Code Explanation

AWS Glue PySpark Job - Code ExplanationThis code implements a production-ready AWS Glue ETL job that processes customer and order data with data quality checks, transformations, and catalog integration.---

High-Level Architecture

The job follows a modular, class-based design with clear separation of concerns:1. **Context Management** - Initialize Spark/Glue environments2. **Configuration** - Load job parameters from YAML3. **Data I/O** - Read from and write to S34. **Data Quality** - Clean and validate data5. **Schema Management** - Apply type-safe schemas6. **SCD2 Tracking** - Manage slowly changing dimensions7. **Aggregation** - Calculate business metrics8. **Catalog Integration** - Register tables in Glue Data Catalog---

Detailed Step-by-Step Explanation

Step 1: Context Initialization (`SparkContextManager`)

Purpose: Set up Spark and Glue runtime environments**How it works:**- Checks if running in test mode (pre-existing Spark session)- In production: Creates SparkContext → GlueContext → SparkSession → Job- Returns all contexts plus a logger for tracking**Key logic:**

```
if hasattr(__builtins__, 'spark'): # Test environment spark =
__builtins__.sparkelse: # Production environment sc = SparkContext.getOrCreate()
glueContext = GlueContext(sc)
```

Step 2: Configuration Loading (`ConfigManager`)

Purpose: Load job parameters from external YAML file with fallback defaults**How it works:**- Tries multiple file paths to locate `config/glue_params.yaml`- If file not found, returns hardcoded default configuration- Configuration includes: - Input/output S3 paths - File formats (CSV, Parquet) - Glue catalog database/table names - Data quality rules - Hudi settings**Key sections in config:**

```
inputs: customer: {source_path, format, options} order: {source_path, format,
options}outputs: customer_curated: {target_path, format, mode}catalog:
database_name: gen_ai_poc_databrickscoe tables: {customer, order, ...}
```

Step 3: Data Reading (`S3DataReader`)

Purpose: Safely read data from S3 with error handling**How it works:**- Uses Spark DataFrameReader with configurable format and options- Normalizes all column names to lowercase for consistency- Returns `None` if read fails (allows graceful error handling)**Example flow:**

```
reader = spark.read.format("csv")reader = reader.option("header", "true")df =
reader.load("s3://<BUCKET_NAME>/customerdata/")df = df.toDF(*[c.lower() for c in
df.columns]) # Normalize columns
```

Step 4: Schema Validation (`SchemaValidator`)

Purpose: Apply strict type-safe schemas to raw data**Defined schemas:****Customer Schema:**- `custid` (String, not null)- `name` (String, not null)- `emailid` (String, not null)- `region` (String, not null)**Order Schema:**- `orderid` (String, not null)- `itemname` (String, not null)- `priceperunit` (Decimal(10,2), not null)- `qty` (Integer, not null)- `date` (String, not null)- `custid` (String, not null)**How schema is applied:**

```
select_exprs = []for field in schema.fields:
select_exprs.append(col(field.name).cast(field.dataType))df =
df.select(*select_exprs)
```

Step 5: Data Cleaning (`DataCleaner`)

Purpose: Remove invalid, null, and duplicate records**Cleaning operations:**1. **Null removal:** - Drops rows with actual NULL values using `df.na.drop()` - Filters out string representations: 'Null', 'NULL', 'null'2. **Duplicate removal:** - Uses `df.dropDuplicates()` to remove exact duplicate rows**Example logic:**

```
df_cleaned = df.na.drop() # Remove actual NULLsfor column in df.columns:
df_cleaned = df_cleaned.filter( (col(column) != 'Null') | col(column).isNull()
)df_cleaned = df_cleaned.dropDuplicates()
```

Step 6: SCD Type 2 Management (`SCD2Manager`)

Purpose: Track historical changes to customer records**SCD2 columns added:**- `isactive` (Boolean) - Current record flag- `startdate` (Timestamp) - When record became active- `enddate` (Timestamp) - When record was superseded (NULL for current)- `opts` (Timestamp) - Operation timestamp for Hudi precombine**How it works:**

```
df_scd2 = df.withColumn("isactive", lit(True)) .withColumn("startdate",
current_timestamp()) .withColumn("enddate", lit(None)) .withColumn("opts",
current_timestamp())
```

Hudi integration:- Writes to Hudi table format for upsert capability- Uses `custid` as record key- Uses `opts` as precombine field (determines latest version)- Enables Hive metastore sync for catalog integration---

Step 7: Data Aggregation (`AggregationEngine`)

Purpose: Calculate customer spending metrics**Calculation steps:**1. **Calculate order amount:**

```
orderamount = priceperunit * qty
```

2. **Join with customer data:** - Inner join on `custid` - Brings in customer `name`3. **Aggregate by customer and date:**

```
df_aggregate = df.groupBy("name", "date")
.agg(sum("orderamount").alias("totalspend"))
```

4. **Round to 2 decimal places:**

```
df_aggregate = df_aggregate.withColumn( "totalspend", round(col("totalspend"), 2)
)
```

Output schema:- `name` (String) - Customer name- `date` (String) - Order date- `totalspend` (Decimal) - Total spending for that customer on that date---

Step 8: Data Writing (`S3DataWriter`)

Purpose: Write processed data to S3 in multiple formats**Supported formats:**- **Parquet** (default) - Columnar, compressed, efficient for analytics- **CSV** - Human-readable, configurable options- **JSON** - Semi-structured format**Write modes:**- `overwrite` - Replace existing data- `append` - Add to existing data**Key features:**- Empty DataFrame check before writing- Optional partitioning by columns- Format-specific options (e.g., CSV header)**Example:**

```
df.write.format("parquet") .mode("overwrite") .partitionBy("region")
.save("s3://<BUCKET_NAME>/curated/customer/")
```

Step 9: Catalog Registration (`CatalogManager`)

Purpose: Register processed data in AWS Glue Data Catalog**How it works:**1. **Create temporary view:**

```
df.createOrReplaceTempView("temp_customer")
```

2. **Create external table:**

```
CREATE TABLE IF NOT EXISTS database.table_name USING parquet LOCATION
's3://<BUCKET_NAME>/path/' AS SELECT * FROM temp_customer
```

Benefits:- Makes data queryable via Athena- Enables schema discovery- Integrates with AWS analytics services---

Step 10: Main Execution Flow (`main()`)

Complete job orchestration:

```
1. Initialize Spark/Glue contexts2. Load configuration from YAML3. Read customer
CSV from S3 (TR-INGEST-001)4. Read order CSV from S3 (TR-INGEST-002)5. Apply
customer schema6. Apply order schema7. Clean customer data (TR-CLEAN-001)8. Clean
order data (TR-CLEAN-002)9. Add SCD2 columns to customer data10. Write customer
data to S3 + catalog (TR-CATALOG-001)11. Write order data to S3 + catalog
(TR-CATALOG-002)12. Calculate customer aggregate spending (TR-AGG-001)13. Write
aggregate data to S3 + catalog14. Write Hudi table for SCD2 tracking15. Commit job
```

Error handling:- Try-except-finally block wraps entire job- Logs errors via Glue logger- Ensures Spark session cleanup- Raises exceptions for job failure tracking---

Key Design Patterns

1. Defensive Programming

- All I/O operations return `None` on failure- Empty DataFrame checks before writes- Multiple config file path attempts

2. Separation of Concerns

- Each class has single responsibility- Static methods for stateless operations- Clear interfaces between components

3. Configuration-Driven

- All paths, formats, options externalized- Easy to modify without code changes- Default fallback configuration

4. Production-Ready Features

- Comprehensive logging at each step- Error handling with context- Test environment detection- Resource cleanup in finally block---

Data Flow Summary

```
Raw CSV (S3) ↓Schema Application (type safety) ↓Data Cleaning (nulls, duplicates) ↓SCD2 Enrichment (customer only) ↓Curated Parquet (S3) ↓Glue Catalog Registration ↓Aggregation (customer spending) ↓Analytics Parquet (S3) ↓Hudi Table (SCD2 tracking)
```

Configuration Placeholders

The code uses these placeholder patterns:- S3 paths: `s3://<BUCKET_NAME>/path/`- Database: `gen_ai_poc_databrickscoe` (should be parameterized)- Table names: `sdlc_wizard_customer`, etc.In production, replace with actual values via:- Glue job parameters- Environment variables- Secrets Manager references